

JavaScript Exercises – Answer Key

1. JavaScript Basics & Setup

- **HTML Setup:**
 - `<script src="main.js"></script>`
- **main.js:**
 - `console.log("Welcome to the Community Portal");`
 - `window.onload = () => alert("Page fully loaded!");`

2. Syntax, Data Types, and Operators

```
const eventName = "Music Fest";
```

```
const eventDate = "2026-07-15";
```

```
let seats = 100;
```

```
console.log(`${eventName} on ${eventDate}, Seats: ${seats}`);
```

```
seats--; // when a user registers
```

3. Conditionals, Loops, and Error Handling

```
events.forEach(event => {
```

```
  if (event.date > Date.now() && event.seats > 0) {
```

```
    console.log(event.name);
```

```
  }
```

```
});
```

```
try {
```

```
  registerUser(eventId);
```

```
} catch (error) {  
  console.error("Registration failed:", error);  
}
```

4. Functions, Scope, Closures, Higher-Order Functions

```
function addEvent(event) { events.push(event); }
```

```
function registerUser(event) { event.seats--; }
```

```
function filterEventsByCategory(category, callback) {  
  return events.filter(e => e.category === category && callback(e));  
}
```

```
// Closure example
```

```
function categoryTracker() {
```

```
  let count = 0;
```

```
  return () => ++count;
```

```
}
```

```
const musicRegistrations = categoryTracker();
```

5. Objects and Prototypes

```
class Event {
```

```
  constructor(name, date, seats) {
```

```
    this.name = name;
```

```
    this.date = date;
```

```
    this.seats = seats;
```

```
}
```

```
}  
Event.prototype.checkAvailability = function() {  
  return this.seats > 0;  
};  
  
console.log(Object.entries(new Event("Workshop", "2026-08-01", 20)));
```

6. Arrays and Methods

```
events.push(new Event("Dance Show", "2026-09-01", 50));  
  
const musicEvents = events.filter(e => e.category === "Music");  
const cards = events.map(e => `Event: ${e.name}`);
```

7. DOM Manipulation

```
const container = document.querySelector("#events");  
const card = document.createElement("div");  
card.textContent = "Community Workshop";  
container.appendChild(card);
```

8. Event Handling

```
document.querySelector("#registerBtn").onclick = () => registerUser(event);  
  
document.querySelector("#categoryFilter").onchange = e =>  
  filterEventsByCategory(e.target.value);  
  
document.querySelector("#search").onkeydown = e =>  
  console.log("Searching:", e.key);
```

9. Async JS, Promises, Async/Await

```
fetch("mockapi/events.json")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

```
async function loadEvents() {
  const res = await fetch("mockapi/events.json");
  const data = await res.json();
  console.log(data);
}
```

10. Modern JavaScript Features

```
function greet(name = "Guest") { console.log(`Hello ${name}`); }
```

```
const { name, date } = event;
const clonedEvents = [...events];
```

11. Working with Forms

```
form.addEventListener("submit", e => {
  e.preventDefault();
  const { name, email, event } = form.elements;
  if (!email.value.includes("@")) {
    alert("Invalid email!");
  }
});
```

12. AJAX & Fetch API

```
fetch("mockapi/register", {  
  method: "POST",  
  body: JSON.stringify(userData)  
})  
  .then(() => alert("Success"))  
  .catch(() => alert("Failed"));  
  
setTimeout(() => console.log("Response delayed"), 2000);
```

13. Debugging and Testing

- Use **Console.log()** at each step.
- Inspect **Network tab** for fetch payload.
- Add breakpoints in **DevTools Sources**.

14. jQuery and Frameworks

```
$("#registerBtn").click(() => alert("Registered!"));  
$("#card").fadeIn().fadeOut();
```

- **Benefit of React/Vue:** Component-based, reusable UI, faster updates.